

计算机体系结构

4. 高速缓存基础

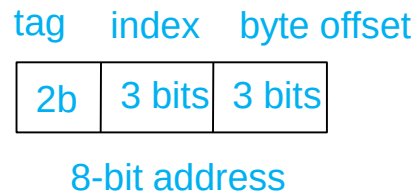
李建华

计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

高速缓存的块和地址映射

- 内存逻辑上被划分为固定大小的块： blocks/lines
- 每个block 会被映射到缓存中的一个确定的**组 (set)**, 由地址中的**index bits**来确定目标组。
 - index用来定位缓存的tag和data store

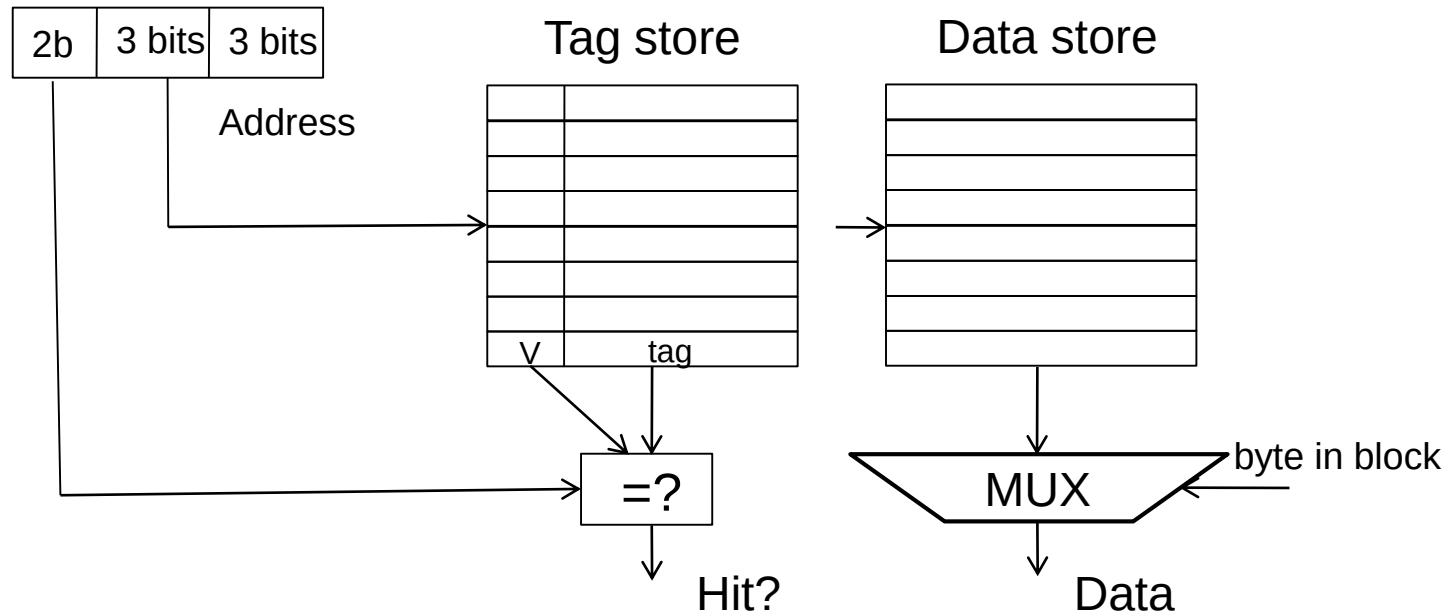


- 缓存的访问:
 - 首先, 用index bits索引到可能的目标tag和data存储;
 - 随后, 检查地址中的tag与tag store中对应的tag是否匹配;
 - 同时, 检查tag存储中的valid bit是否有效;

直接相联缓存

- 假定一个字节寻址的存储器配置: 256 bytes, 8-byte/block $\rightarrow$ 32 blocks
- 假定高速缓存配置: 64 bytes $\rightarrow$ 8 blocks
 - 直接相联: 一个block只能保存到**唯一的**一个位置

tag index byte in block



- **问题:** 具有相同index bits的block会映射到相同的缓存块, 从而发生冲突。

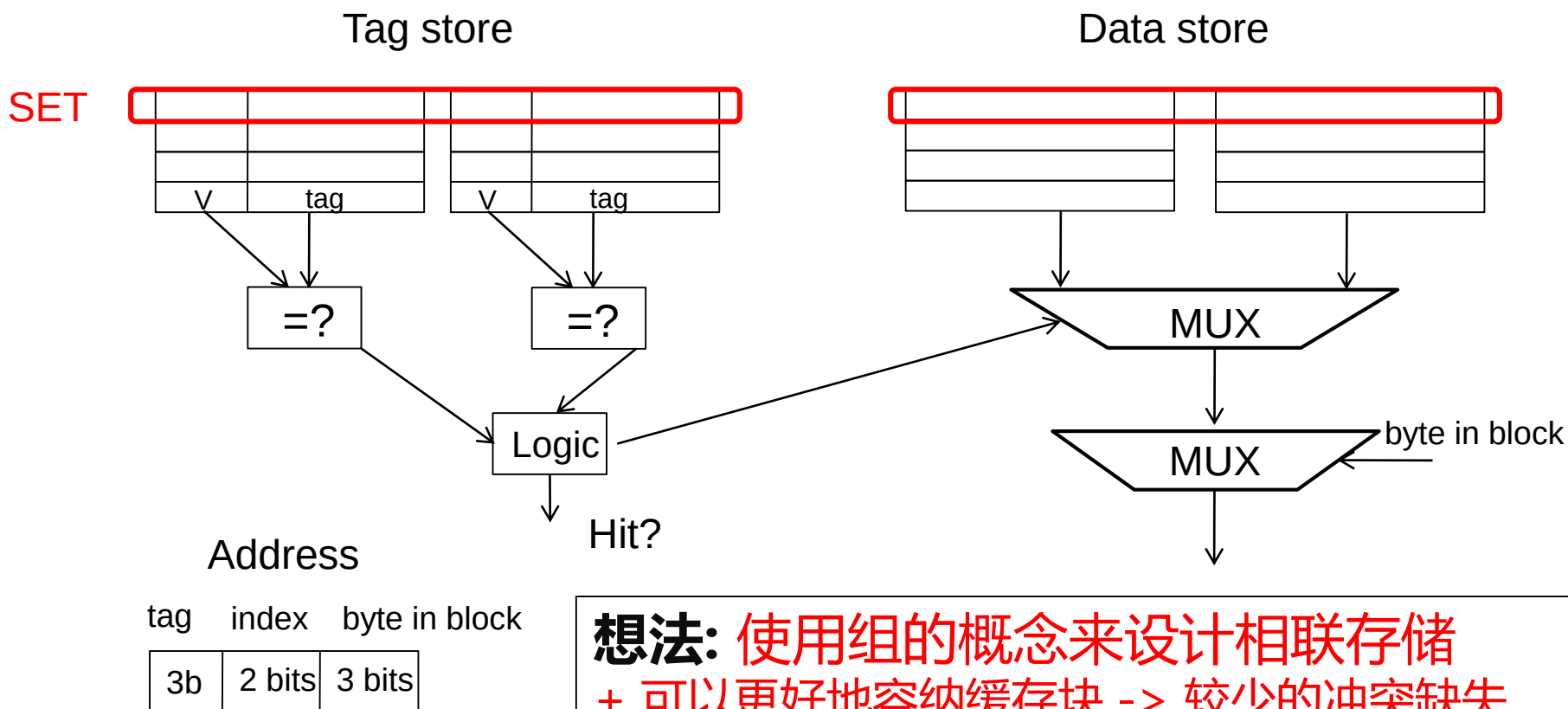
直接相联缓存

- **直接相联缓存:** 存储器中的2个block映射到缓存中的相同index所确定的同一个位置 — 不能同时容纳2个block
 - 同一个索引 → 同一个缓存位置 . . .
- 当循环访问超过1个block (均具有相同的index) , 就会导致缓存的命中率为0。
 - 假定地址A和B具有相同的index bits, 不同的tag bits
 - 访问A, B, A, B, A, B, A, B, ... → index相同会导致冲突
 - 所有的访问都是缺失 (叫做: conflict misses)
 - 这个现象叫做**乒乓效应 (ping-pong effect)**

What's the problem?

2路组相联缓存

- 地址0和8在上述直接相联缓存里总是冲突
- 与其使用1列8个块的结构, 何不尝试使用2列结构?

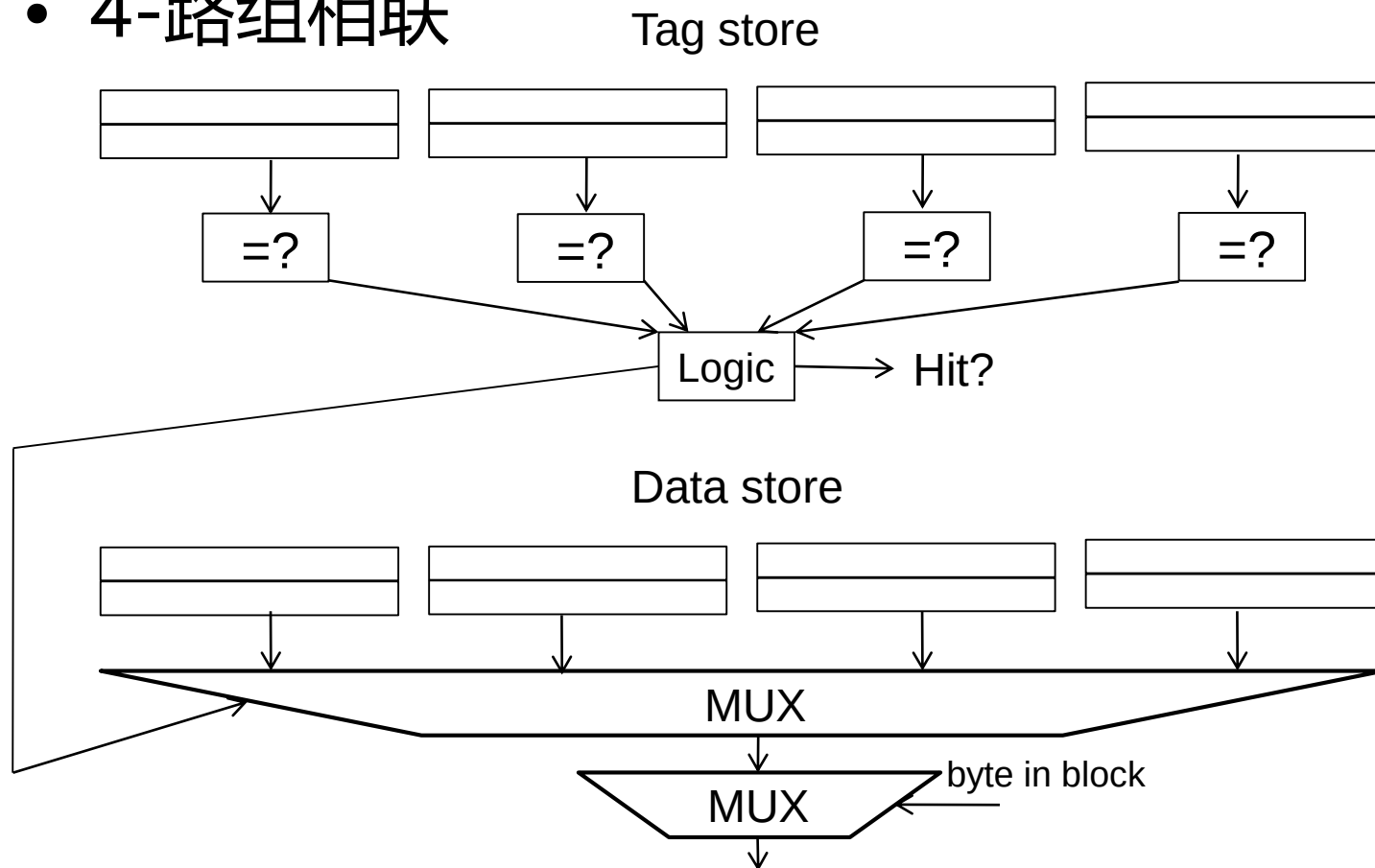


想法: 使用组的概念来设计相联存储

- + 可以更好地容纳缓存块 -> 较少的冲突缺失
- 更加复杂, 访问速度变慢, tag所需存储增加

4路组相联

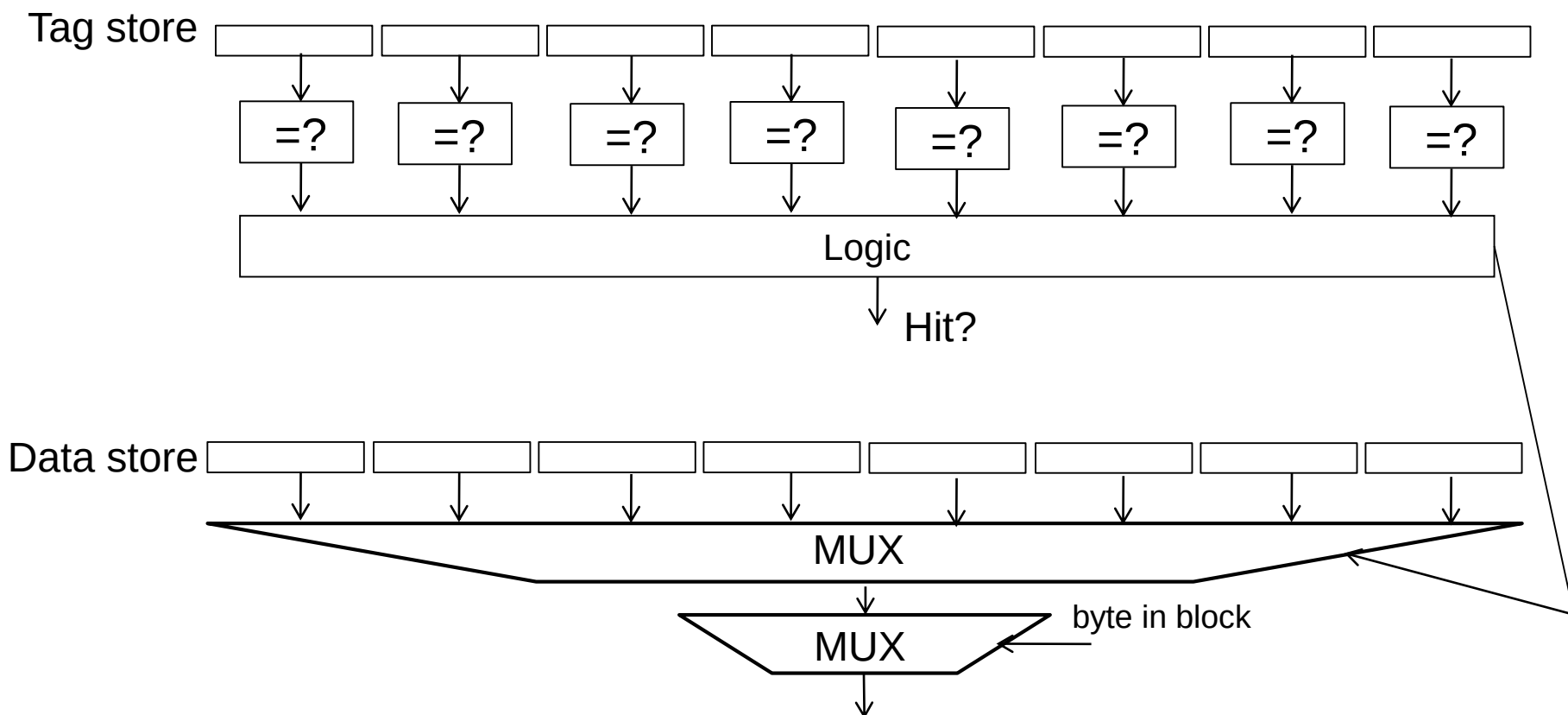
- 4-路组相联



- + 发生冲突（失效）的概率进一步降低
- 更多的tag比较器，以及更宽的数据mux，更大的tag存储

全相联映射

- 全相联映射缓存
 - 每个block可以放置到缓存中的任何位置



组相联缓存的主要问题

- 可以认为缓存组中每个block均具有 “priority”
 - Priority是block在LRU stack中的位置
- 有三个时机，可以调整block的priority：
 - Insertion: 当访问一个缺失的block时
 - 是否将该block放入缓存?
 - 即将导入缓存的block放到什么位置?
 - Promotion: 当在cache命中一个block时
 - 是否 并 如何调整 block priority?
 - **Eviction**: 当访问缓存发生缺失且目标set已满
 - 将哪一个block进行evict? 如何调整priority?

Replacement Policy

- 当发生缓存缺失并且目标set已满的时候，需要调用Replacement Policy 来替换block
 - 优先使用set内的任何 invalid block
 - 若set内所有block均valid，询问replacement策略：
 - Random
 - FIFO
 - **LRU: Least recently used (how to implement?)**
 - NRU: Not most recently used
 - LFU: Least frequently used?
 - Least costly to re-fetch?
 - Why would memory accesses have different cost?
 - Hybrid replacement policies
 - Optimal replacement policy?

Details: LRU算法的实现

- **想法:** 将组内的least recently accessed block踢出缓存。【How to?】
- **问题:** 需要维持组内block的历史访问顺序
- 对于2-路组相联缓存:
 - 如何实现LRU算法?
- 对于4-路组相联缓存:
 - 如何高效地实现LRU算法?
 - 一个组内有4个块, 有多少种可能的访问顺序?
 - 需要多少个bits来记录访问顺序?
 - How to choose LRU victim?

我们来看关于LRU替换策略的一个示例

Approximations of LRU

- 大多数的当代处理器没有实现高相联度缓存的“true LRU”策略
- Why?
 - True LRU 过于复杂
 - LRU 也只是用来预测局部性信息
 - 其未必一直或者一定是最好的策略
- 其它方案:
 - Not MRU (not most recently used)
 - Hierarchical LRU
 - Victim-NextVictim Replacement
 -

替换算法示例：LRU or Random?

- LRU vs. Random: 哪一个性能更好?
 - 考虑: 4路组相联缓存, 循环访问: A, B, C, D, E
 - LRU策略所获得的命中率为0
- Set thrashing: 当所运行的程序在一个set内的“working set” 大于相联度
 - 这时候随机策略可能更好, 因为可避免抖动。
- 实际情况下, 哪个替换策略更好呢?
 - 取决于workload的特征和缓存的配置
 - 其实, LRU和Random的平均命中率类似
- **更好的方案: LRU和Random混合策略**
 - 到底选择哪一个方案? Set sampling
 - See: Qureshi et al., “A Case for MLP-Aware Cache Replacement ”, ISCA 2006.

有没有最优的替换策略?

- Belady's OPT (理论上最优的策略)
 - 替换将来最久/最远被访问的block
 - Belady, "A study of replacement algorithms for a virtual-storage computer," IBM Systems Journal, 1966.
 - 如何实现这个最优的算法?
- 该算法能获得最小的缺失率吗? [有没有其它指标?]
- 该算法能获得最小的程序执行时间吗?
 - 不一定, 缓存中block的缺失延迟是不同的, 非均匀的!
 - 主要原因: 远程和本地缓存访问 以及 缺失的重叠处理。
- Qureshi et al. "[A Case for MLP-Aware Cache Replacement](#)," in ISCA 2006.

有没有最优的替换策略？

Abstract:

Performance loss due to long-latency memory accesses can be reduced by servicing multiple memory accesses concurrently. The notion of generating and servicing long-latency cache misses in parallel is called Memory Level Parallelism (MLP). MLP is not uniform across cache misses - some misses occur in isolation while some occur in parallel with other misses. Isolated misses are more costly on performance than parallel misses. However, traditional cache replacement is not aware of the MLP-dependent cost differential between different misses. Cache replacement, if made MLP-aware, can improve performance by reducing the number of performance-critical isolated misses. This paper makes two key contributions. First, it proposes a framework for MLP-aware cache replacement by using a runtime technique to compute the MLP-based cost for each cache miss. It then describes a simple cache replacement mechanism that takes both MLP-based cost and recency into account. Second, it proposes a novel, low-hardware overhead mechanism called Sampling Based Adaptive Replacement (SBAR), to dynamically choose between an MLP-aware and a traditional replacement policy, depending on which one is more effective at reducing the number of memory related stalls. Evaluations with the SPEC CPU2000 benchmarks show that MLP-aware cache replacement can improve performance by as much as 23%.

Published in: 33rd International Symposium on Computer Architecture (ISCA'06)

- Qureshi et al. "[A Case for MLP-Aware Cache Replacement](#)", in ISCA 2006.

Bonus: Cache vs Main Memory

- Main Memory (DRAM) 是磁盘/SSD的缓存
 - 通常由Virtual Memory子系统软件来管理
 - 需要有辅助的硬件：TLB, MMU
 - DRAM和二级存储的交换粒度：Page（页面）
- Page的替换和Block的替换非常类似
- Page Table就是物理存储所对应的 “tag store”
- 二者之间的区别是什么？
 - 访问速度： cache vs. physical memory
 - Block的数量： cache vs. physical memory
 - 寻找替换对象（victim）时可以容忍的时间
 - 管理方式： hardware versus software

Tag Store中都保存了什么?

- Valid bit – 该block保存的数据是否有效
- Tag – 该block保存的数据的ID (Key)
- Replacement policy bits – priority信息
- Dirty bit – 表明该block是否被修改过
 - Used in Write-Back Cache
- Anything else?
 - 例如, 多核的一致性协议状态。



这些不是数据本身的数据,
叫做元数据 (metadata)。

写操作的处理 (1)

- 何时将修改了的数据写到下一级缓存？
 - Write-through: 当前缓存写发生的同时，写到下一级缓存；
 - Write-back: 当block从当前缓存被替换的时候写到下一级缓存。
- Write-back
 - + 在被替换之前，可以合并对同一个block的多次写，从而减少缓存层级之间的带宽消耗，同时可节省能耗。
 - 标签存储中需要一个额外的dirty bit来标明 block是否被更新
- Write-through
 - + 其实现简单，逻辑和硬件都较简单。
 - + 所有Cache层级中的数据都是最新的，简化一致性协议的实现。
 - 与写回策略相比，需要消耗更多的带宽。

写操作的处理 (2)

- 当发生写缺失时，是否在缓存中分配一个block？
 - 写分配 (write allocate) : Yes
 - 写不分配 (write non-allocate) : No
- Write allocate：
 - + 可以聚合写操作，而不需要每次都写到下一级存储。
 - + 更加简单：因为写可以和读一样进行处理
 - 需要在层级之间传输整个缓存块，可能带来问题 ???
- Write non-allocate：
 - + 若写操作的局部性低，可节省缓存空间 (可能具有更好的缓存命中率)
 - 若局部性好，则性能不如write allocate

Harvard or Princeton?

- 合并架构（Princeton Architecture）的优缺点：
 - + 指令和数据可以共享缓存空间: 不需要分离设计中空间的overprovisioning（超量配置）
 - 指令和数据可能相互竞争，没有私有空间的性能保障；
 - 流水线中指令和数据在不同的阶段访问，合并的架构可能会导致结构冲突（structural hazard）。
- 分离架构（Harvard Architecture）：
 - 主要是为了避免流水线的结构冲突
- 商用CPU的一级缓存绝大多数采取分离架构
- 二级和其它级别的缓存通常采用合并的方案，以提升缓存空间的利用率。

流水线设计中的多级缓存

- 一级缓存 (instruction and data)
 - 设计的很多决定受cycle time影响
 - Small, lower associativity
 - Tag store和data store通常并行访问
- 二级缓存
 - 设计的决定需要均衡: hit rate 和 access latency
 - 通常large, highly associative; 延迟不是最重要的。
 - Tag store和data store通常顺序访问
- 层级之间的顺序和并行访问
 - Serial: 一级缓存缺失的时候, 才访问二级;
 - 二级缓存所看到的访问请求和一级所看到的不同:
 - 一级缓存像是一个访问过滤器 (过滤掉了一些局部性)
 - 所以, 不同缓存级别所适用的管理策略不同。

高速缓存的参数与性能

- Cache size – 缓存的大小
- Block size – 缓存块的大小
- Associativity – 缓存的相联度
- Replacement policy – 缓存的替换策略
- Placement policy – 缓存的放置策略
 - 初始fill 时的Placement放置
 - 后续hit 时的Promotion调整

高速缓存的性能优化

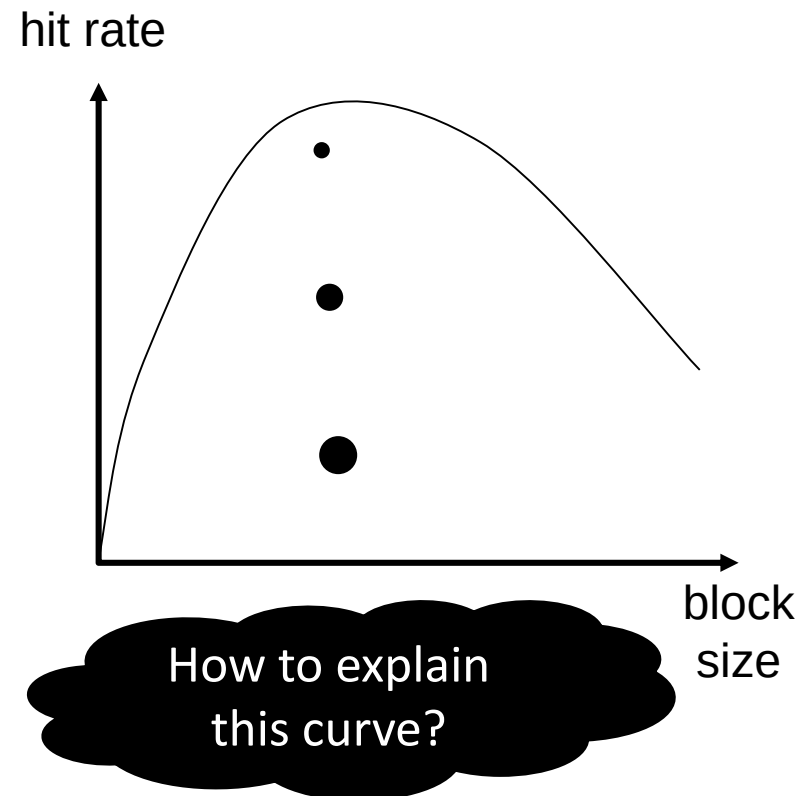
Ten advanced optimizations

1. Small and simple first-level caches to reduce hit time and power
2. Way prediction to reduce hit time
3. Pipelined access and multi-banked caches to increase bandwidth
4. Non-blocking caches to increase cache bandwidth
5. Critical word first and Early restart to reduce miss penalty
6. Merging write buffer to reduce miss penalty
7. Compiler optimizations to reduce miss rate
8. Hardware prefetching to reduce miss penalty or miss rate
9. Compiler-controlled prefetching to reduce miss penalty or miss rate
10. Using HBM to extend the memory hierarchy

1. Large block size

- Block size是与地址中tag相关联的一个参数
 - 不一定非要是存储层级之间数据传输的基本粒度
 - Sub-blocking: 将一个block划分为多个pieces (每个都配置 valid bit) -> 可以提升写操作性能

- 太小的block
 - 不能很好地挖掘空间局部性
 - 存储tag所需的空間开销较大
- 太大的block
 - 容量相同, Block数量过少
 - 挖掘时间局部性难度增大
 - 当空间局部性差的时候
 - 浪费空间浪费带宽, 浪费功耗



Miss rate vs block size

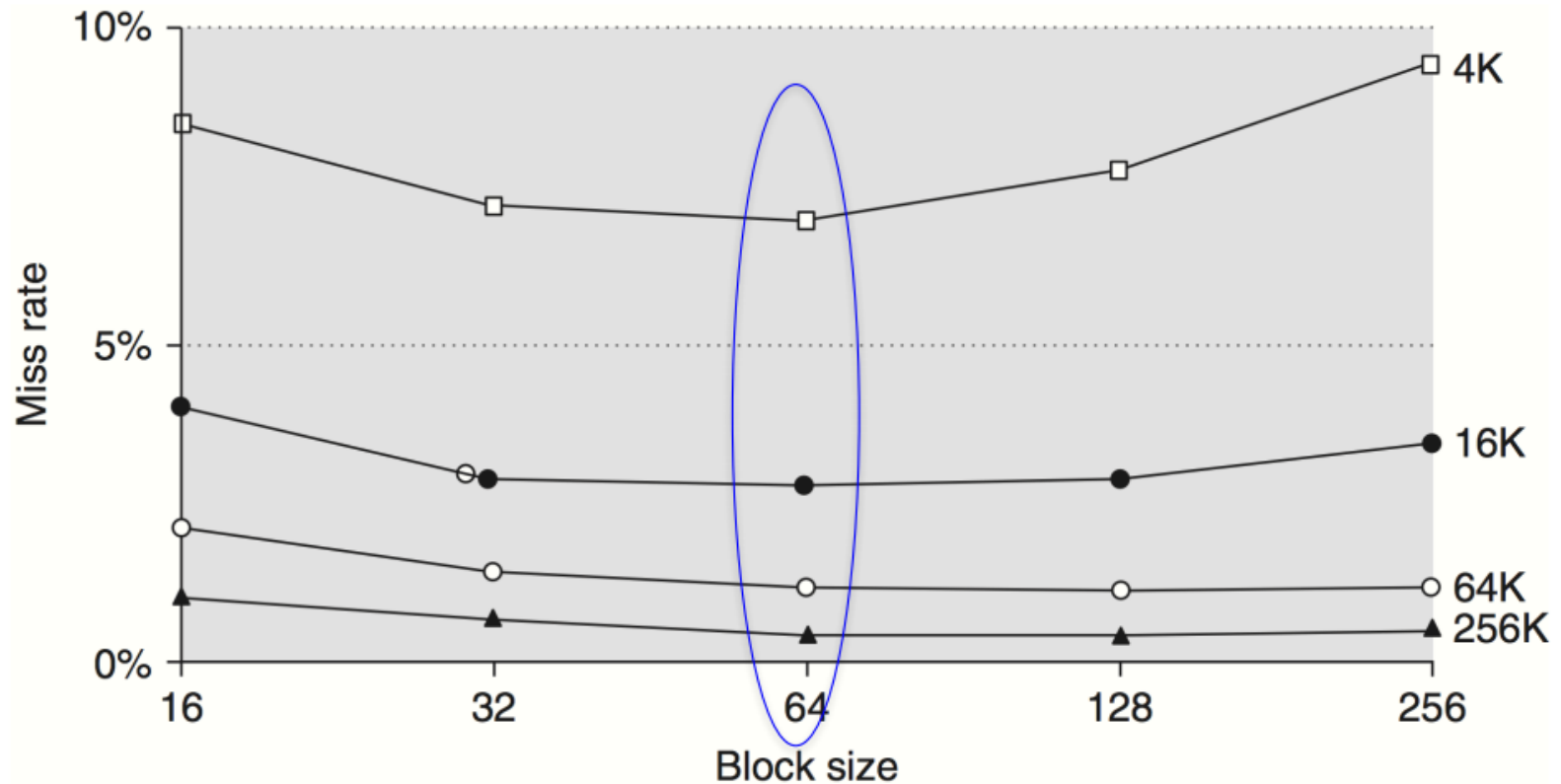


Figure B.10 Miss rate versus block size for five different-sized caches. Note that miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. [Figure B.11](#) shows the data used to plot these lines. Unfortunately, SPEC2000 traces would take too long if block size were included, so

Demo

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Figure B.11 Actual miss rate versus block size for the five different-sized caches in **Figure B.10**. Note that for a 4 KB cache, 256-byte blocks have a higher miss rate than 32-byte blocks. In this example, the cache would have to be 256 KB in order for a 256-byte block to decrease misses.

Demo

Example Figure B.11 shows the actual miss rates plotted in Figure B.10. Assume the memory system takes 80 clock cycles of overhead and then delivers 16 bytes every 2 clock cycles. Thus, it can supply 16 bytes in 82 clock cycles, 32 bytes in 84 clock cycles, and so on. Which block size has the smallest average memory access time for each cache size in Figure B.11?

Answer Average memory access time is

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

If we assume the hit time is 1 clock cycle independent of block size, then the access time for a 16-byte block in a 4 KB cache is

$$\text{Average memory access time} = 1 + (8.57\% \times 82) = 8.027 \text{ clock cycles}$$

and for a 256-byte block in a 256 KB cache the average memory access time is

$$\text{Average memory access time} = 1 + (0.49\% \times 112) = 1.549 \text{ clock cycles}$$

Demo

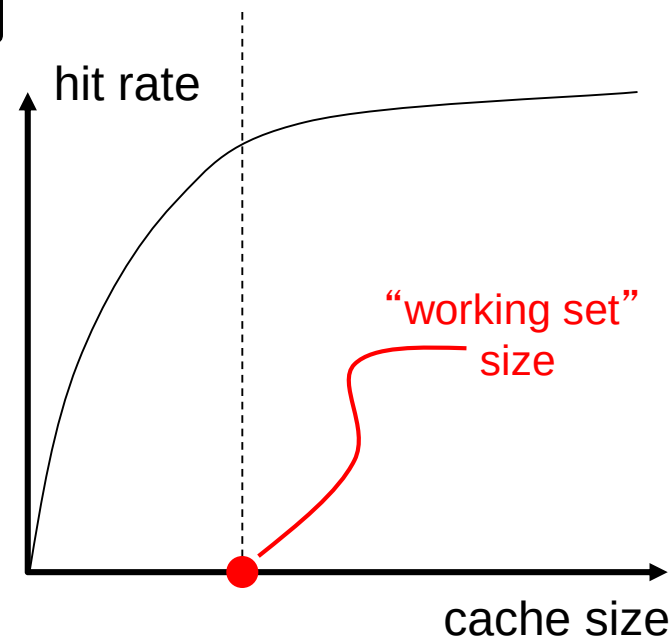
Block size	Miss penalty	Cache size			
		4K	16K	64K	256K
16	82	8.027	4.231	2.673	1.894
32	84	7.082	3.411	2.134	1.588
64	88	7.160	3.323	1.933	1.449
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

Figure B.12 Average memory access time versus block size for five different-sized caches in Figure B.10. Block sizes of 32 and 64 bytes dominate. The smallest average time per cache size is boldfaced.

How to calculate AMAT?

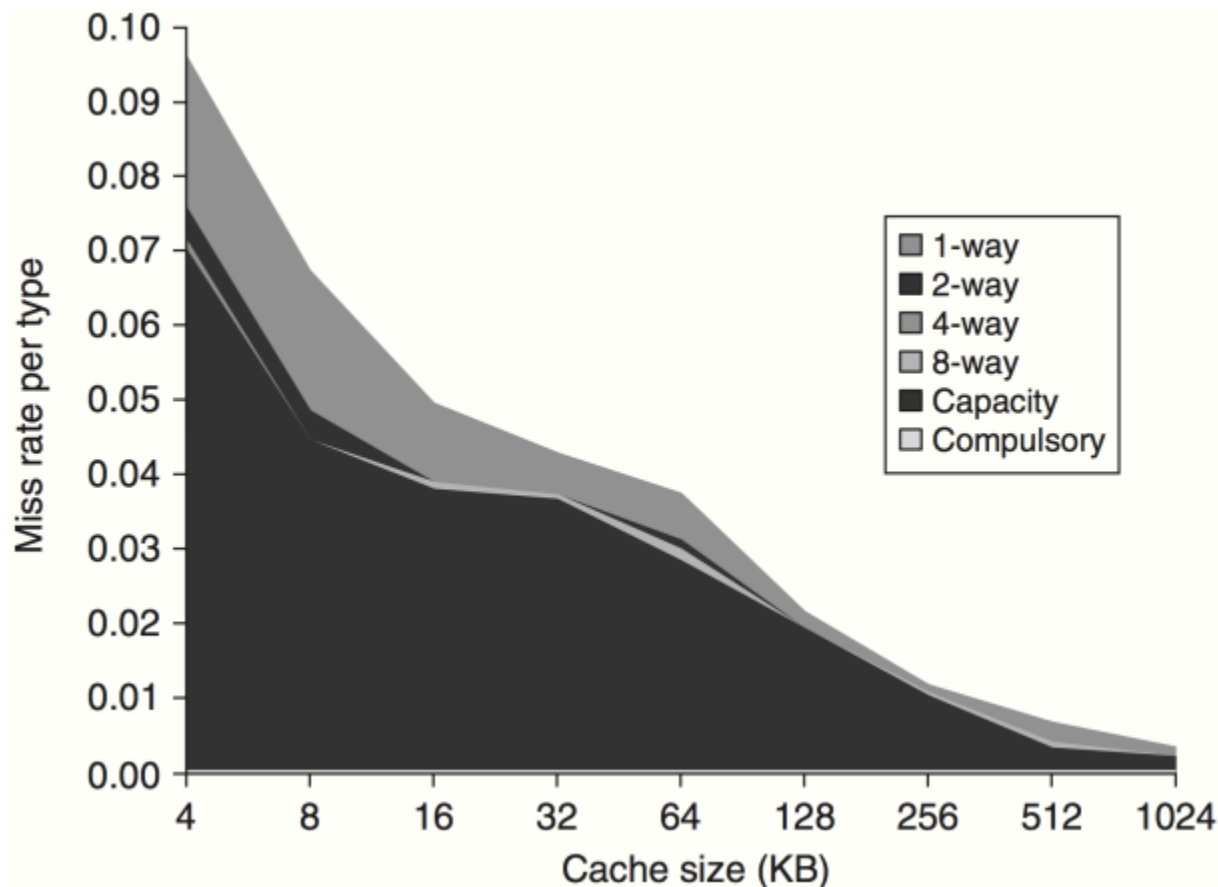
2. Large cache size

- 缓存的大小: 总的容量 (不包括Tag所占空间)
 - 容量越大就能更好地挖掘利用时间局部性
 - 但是, 容量越大 **不等价于** 性能越好。
- 容量太大对访问延迟有负面影响
 - 因为: **smaller is faster**
 - 访问时间可能影响关键路径
- 容量太小影响局部性的挖掘
 - 不能很好地挖掘时间局部性
 - 有用的数据可能会被频繁替换
- Working set**: 应用程序访问的所有数据的合集
 - 注意**: 是在一个时间窗口内



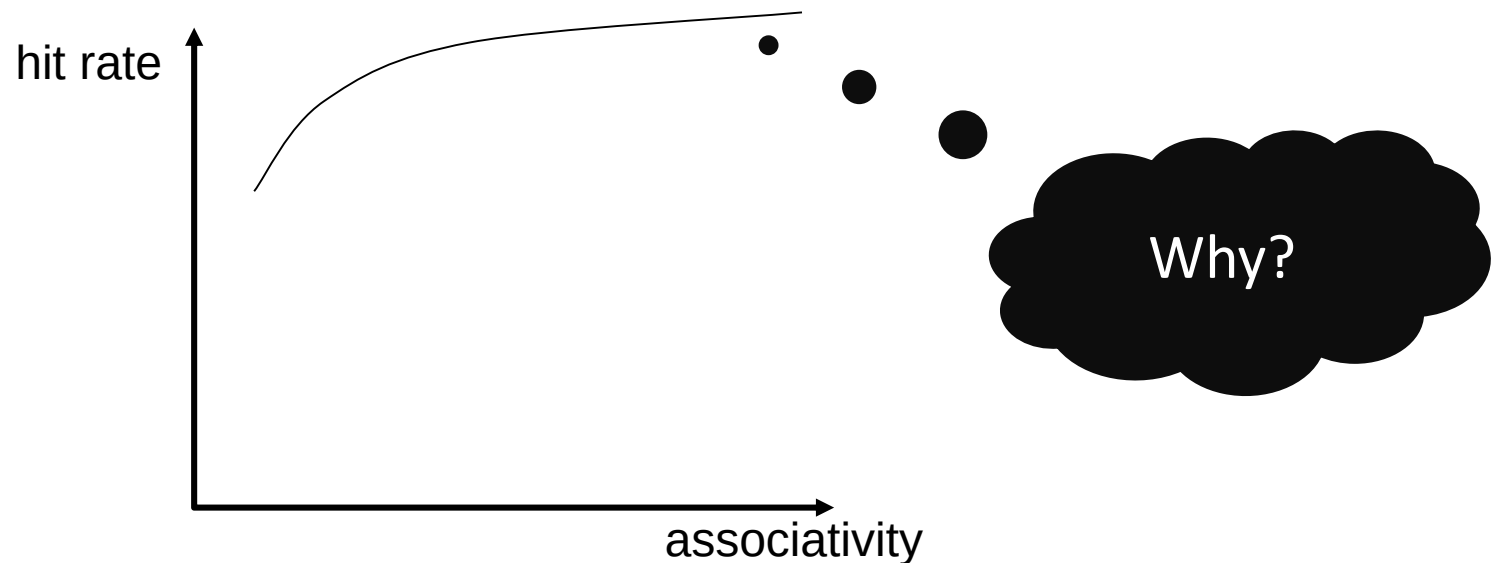
增大Cache Size的影响

- 较大的缓存，可能命中时间会增加，且开销更大；
- 目前的趋势：二级缓存和三级缓存的容量越来越大



3. High associativity

- **相联度的大小:** 多少个block可以放置到相同的组?
- 高相联度缓存的优点和缺点:
 - + 较高的缓存命中率
 - **较慢的访问速度** (命中延迟 和 数据访问延迟)
 - **较大的硬件开销** (需要更多的比较器)
- 此外, 高相联度带来的收益有一个衰减趋势。



Associativity

- 低相联度缓存的特性
 - 硬件开销小
 - 访问延迟低
 - 对L1来说尤其重要
 - 缺失率高
- 相联度需要是2的整数次幂吗?
 - 不是必须的，比如有12-路的商用缓存设计

Demo

Example Assume that higher associativity would increase the clock cycle time as listed below:

$$\text{Clock cycle time}_{2\text{-way}} = 1.36 \times \text{Clock cycle time}_{1\text{-way}}$$

$$\text{Clock cycle time}_{4\text{-way}} = 1.44 \times \text{Clock cycle time}_{1\text{-way}}$$

$$\text{Clock cycle time}_{8\text{-way}} = 1.52 \times \text{Clock cycle time}_{1\text{-way}}$$

Assume that the hit time is 1 clock cycle, that the miss penalty for the direct-mapped case is 25 clock cycles to a level 2 cache (see next subsection) that never misses, and that the miss penalty need not be rounded to an integral number of clock cycles. Using [Figure B.8](#) for miss rates, for which cache sizes are each of these three statements true?

$$\text{Average memory access time}_{8\text{-way}} < \text{Average memory access time}_{4\text{-way}}$$

$$\text{Average memory access time}_{4\text{-way}} < \text{Average memory access time}_{2\text{-way}}$$

$$\text{Average memory access time}_{2\text{-way}} < \text{Average memory access time}_{1\text{-way}}$$

Cache size (KB)	Associativity			
	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62

Demo

Answer Average memory access time for each associativity is

$$\begin{aligned}\text{Average memory access time}_{8\text{-way}} &= \text{Hit time}_{8\text{-way}} + \text{Miss rate}_{8\text{-way}} \times \text{Miss penalty}_{8\text{-way}} \\ &= 1.52 + \text{Miss rate}_{8\text{-way}} \times 25\end{aligned}$$

$$\text{Average memory access time}_{4\text{-way}} = 1.44 + \text{Miss rate}_{4\text{-way}} \times 25$$

Cache size (KB)	Associativity			
	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62
16	2.23	2.40	2.46	2.53
32	2.06	2.30	2.37	2.45
64	1.92	2.14	2.18	2.25
128	1.52	1.84	1.92	2.00
256	1.32	1.66	1.74	1.82
512	1.20	1.55	1.59	1.66

Figure B.13 Average memory access time using miss rates in Figure B.8 for parameters in the example. Boldface type means that this time is higher than the number to the left, that is, higher associativity *increases* average memory access time.